

공학석사학위논문

조합 최적화를 위한 확산 모델

Diffusion Models for Combinatorial Optimization

2026 년 2 월

서울대학교 대학원

산업공학과

최 장 혁

조합 최적화를 위한 확산 모델

Diffusion Models for Combinatorial Optimization

지도교수 지도교수

이 논문을 공학석사 학위논문으로 제출함

2026 년 1 월

서울대학교 대학원

산업공학과

최 장 혁

최장혁의 공학석사 학위논문을 인준함

2026 년 2 월

위원장 위원장 성함 (인)

부위원장 지도교수 성함 (인)

위원 위원 성함 (인)

초록

본 논문에서는 조합 최적화 문제를 확산 기반 신경망 해법으로 다룬다. 확산 모델이 생성하는 열지도(heatmap)를 문제 제약을 만족하는 해로 디코딩하는 과정이 성능의 핵심 병목이며, 이를 완화하기 위해 CADO 스타일의 디코더 인식 강화학습 미세조정을 적용한다. TSP, CVRP, MDVRP에 대한 실험을 통해 제안 방법의 최적성 간격(gap), 계산 비용, 및 주요 절제(ablation) 결과를 보고한다.

주요어: 조합최적화, 확산모델, 강화학습, 신경망 조합최적화, 차량 경로 문제

학번: 2020-12053

목차

초록	i
목차	iv
표 목차	v
그림 목차	vi
제 1 장 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Contributions	3
1.4 Thesis Outline	3
제 2 장 Literature Review	4
2.1 Optimization Problems	4
2.1.1 Combinatorial Optimization	4
2.1.2 Traveling Salesman Problem	5
2.1.3 Capacitated Vehicle Routing Problem	5
2.1.4 Multi-Depot Vehicle Routing Problem	7
2.2 Classical and Learning-Based Solvers	8
2.2.1 Exact Solvers	9

2.2.2	Relaxation and Decomposition Methods	9
2.2.3	Heuristics and Metaheuristics	10
2.2.4	Learning-Assisted Solvers	11
2.3	Neural Network Methodologies	11
2.3.1	Graph Neural Networks	11
2.3.2	Anisotropic Graph Neural Networks	12
2.4	Diffusion Models for COP	13
2.4.1	DIFUSCO	13
2.5	Reinforcement Learning Fine-Tuning	16
2.5.1	CADO	16
2.5.2	MDP Formulation	17
2.5.3	Reward Strategies	18
2.5.4	Hybrid Fine-Tuning	18
제 3 장	Methodology	20
3.1	Overview	20
3.2	Model Architecture	21
3.3	Supervised Training	22
3.4	CADO Fine-Tuning	22
3.5	Decoding and Evaluation	23
3.6	Implementation	24
제 4 장	Experiments	26
4.1	Experimental Setup	26

4.2	Datasets	26
4.3	Baselines and Metrics	27
4.4	Main Results	28
4.5	Ablations	29
4.6	Discussion	31
제 5 장	Conclusion	33
5.1	Summary	33
5.2	Limitations	34
5.3	Future Work	35
제 A 장	Implementation Details	37
A.1	Configuration Reference	37
A.2	Hyperparameter Tables	37
A.3	Additional Plots	37
	참고문헌	38
	Abstract	40
	감사의 글	41

표 목차

표 4.1	Summary of main experimental results. Lower gap is better; higher MDVRP assignment accuracy is better.	28
-------	--	----

그림 목차

그림 3.1	Overview of the proposed training and evaluation pipeline for diffusion-based neural solvers. The pipeline consists of supervised denoising training followed by optional reinforcement learning (CADO-style) fine-tuning.	20
그림 4.1	TSP-50 validation gap during the extended CADO run. The dashed line shows the supervised checkpoint used to initialize fine-tuning.	29
그림 4.2	CVRP-50 validation gap for supervised decoding variants and CADO fine-tuning. 2-opt improves the best supervised checkpoint, while CADO reaches a lower best gap but later collapses.	30
그림 4.3	MDVRP-50 assignment accuracy and supervised loss. The loss decreases steadily, but assignment accuracy degrades after the first validation point.	31

제 1 장 Introduction

1.1 Motivation

Combinatorial optimization (CO) is a fundamental field in computer science and operations research, encompassing the search for an optimal solution from a finite but often exponentially large set of candidates. Classic examples include the traveling salesman problem (TSP), the maximum independent set (MIS), and the vehicle routing problem (VRP). These problems are NP-hard, meaning they are believed to be intractable in polynomial time, and no known algorithm can solve them efficiently at scale.

Traditionally, CO problems have been handled by exact solvers (e.g., Concorde and Gurobi) [1, 4], approximation algorithms, and hand-crafted heuristics such as LKH and local search methods like 2-opt [3, 5, 7]. However, these methods have significant drawbacks, often demanding substantial domain expertise, careful engineering, or considerable computational resources. These limitations have motivated the search for data-driven alternatives. In recent years, neural combinatorial optimization (NCO) has emerged as a promising approach to address the constraints of traditional methods [2].

As the field of NCO has progressed, various architectures have been proposed. In this paper, we utilize a diffusion-based architecture modeled with a Bernoulli distribution to solve CO problems for two primary reasons: 1) Diffusion-based ar-

chitectures can capture high-dimensional, multi-modal distributions, evaluating solution probabilities simultaneously. Due to the inherent complexity of CO problems, this multi-modality is crucial for capturing the underlying problem structure. 2) The diffusion process iteratively refines its output, a mechanism that conceptually mirrors the iterative improvement approaches of classical solvers. Consequently, this research investigates methods to enhance the capabilities of diffusion-based models in solving CO problems.

1.2 Problem Statement

Diffusion-based models generate outputs in the form of a heatmap, representing probability distributions over the solution space. Specifically, discrete diffusion models exploit a Bernoulli distribution. To yield a final, valid solution, this heatmap must be decoded subject to hard problem constraints. However, this decoding process is a primary bottleneck that degrades model performance, as the models are traditionally oblivious to the non-differentiable decoding mechanism and the problem’s strict constraints [11]. In CADO, the authors propose a decoder-aware reinforcement learning approach to align heatmap generation explicitly with the decoding process.

Although CADO successfully introduces decoding awareness to heatmap-based models, its application is restricted to the TSP and MIS, which feature relatively simpler structural constraints compared to other CO problems. In this paper, we focus on capacitated vehicle routing problems (CVRPs) and multi-depot vehicle routing problems (MDVRPs). These variants possess significantly more complex constraints within their structure and represent critical challenges in real-world logistics and

supply chain management.

1.3 Contributions

- Reproduce and extend *DifusCO*-style diffusion solvers for TSP and CVRP.
- Implement *CADO*-style hybrid fine-tuning and REINFORCE on top of supervised checkpoints.
- Empirical comparison: optimality gap, wall-clock cost, and ablations (reward mode, entropy, denoising steps).

1.4 Thesis Outline

제 2 장 Literature Review

2.1 Optimization Problems

2.1.1 Combinatorial Optimization

This paper follows the conventional notation for combinatorial optimization introduced by Papadimitriou and Steiglitz [8]. The space of candidate solutions for a CO problem instance s is denoted by $\mathcal{X}_s = \{0, 1\}^N$, and the objective function is denoted by $c_s : \mathcal{X}_s \rightarrow \mathbb{R}$ for a solution $x \in \mathcal{X}_s$. Given a cost function $c_s(\cdot)$, the goal is to find the optimal solution x^{s*} for the given instance s :

$$x^{s*} = \arg \min_{x \in \mathcal{X}_s} c_s(x).$$

Because enumerating all possible solution sets for a CO problem is computationally intractable, exact methods often relax the binary condition, solve a linear program (LP), and subsequently branch on fractional solution components. This approach is known as branch and bound (B&B). While B&B avoids enumerating entirely unnecessary solution sets, the number of branches can still grow exponentially in the worst case.

2.1.2 Traveling Salesman Problem

Let $G = (V, E)$ be a complete undirected graph with $|V| = n$ cities and edge costs $c_{ij} \geq 0$ for all $\{i, j\} \in E$. Define binary decision variables

$$x_{ij} = \begin{cases} 1, & \text{if the tour uses edge } \{i, j\}, \\ 0, & \text{otherwise.} \end{cases}$$

The traveling salesman problem (TSP) can be formulated as the following integer program:

$$\begin{aligned} \min_x \quad & \sum_{\{i,j\} \in E} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j \in V \setminus \{i\}} x_{ij} = 2, \quad \forall i \in V \quad (\text{degree constraints}) \\ & \sum_{\{i,j\} \in E(S)} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| \leq n - 1 \quad (\text{subtour elimination}) \\ & x_{ij} \in \{0, 1\}, \quad \forall \{i, j\} \in E. \end{aligned}$$

Here, $E(S) = \{\{i, j\} \in E : i \in S, j \in S\}$ denotes the set of edges with both endpoints in subset S . The subtour elimination constraints enforce that the selected edges form a single Hamiltonian cycle visiting every city exactly once, rather than multiple disjoint cycles.

2.1.3 Capacitated Vehicle Routing Problem

Let $G = (V, E)$ be a complete graph where $V = \{0, 1, \dots, n\}$, with depot 0 and customers $1, \dots, n$. Each customer $i \in \{1, \dots, n\}$ has demand $q_i > 0$, and $c_{ij} \geq 0$

is the travel cost between nodes i and j . A fleet of at most m identical vehicles is available, each with capacity Q . Define binary decision variables

$$x_{ij} = \begin{cases} 1, & \text{if a vehicle travels directly from } i \text{ to } j, \\ 0, & \text{otherwise,} \end{cases} \quad \forall i \neq j, i, j \in V,$$

and continuous load variables u_i for $i \in \{1, \dots, n\}$ denoting the cumulative load delivered upon arrival at customer i .

The capacitated vehicle routing problem (CVRP) can be formulated as the following mixed-integer program:

$$\begin{aligned} \min_{x, u} \quad & \sum_{i \in V} \sum_{j \in V \setminus \{i\}} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j \in V \setminus \{i\}} x_{ij} = 1, & \forall i \in V \setminus \{0\} \quad (\text{each customer departs once}) \\ & \sum_{i \in V \setminus \{j\}} x_{ij} = 1, & \forall j \in V \setminus \{0\} \quad (\text{each customer arrives once}) \\ & \sum_{j \in V \setminus \{0\}} x_{0j} \leq m, & (\text{number of routes}) \\ & \sum_{i \in V \setminus \{0\}} x_{i0} = \sum_{j \in V \setminus \{0\}} x_{0j}, & (\text{flow conservation at depot}) \\ & q_i \leq u_i \leq Q, & \forall i \in V \setminus \{0\} \quad (\text{capacity bounds}) \\ & u_i - u_j + Qx_{ij} \leq Q - q_j, & \forall i, j \in V \setminus \{0\}, i \neq j \quad (\text{MTZ}) \\ & x_{ij} \in \{0, 1\}, & \forall i \neq j, i, j \in V. \end{aligned}$$

The Miller–Tucker–Zemlin (MTZ) constraints enforce route connectivity and eliminate subtours while simultaneously ensuring that vehicle capacity is not exceeded.

2.1.4 Multi-Depot Vehicle Routing Problem

Let $G = (V, E)$ be a complete graph containing multiple depots and customers. Let $D = \{0, 1, \dots, d\}$ be the set of depots and $C = \{d + 1, \dots, d + n\}$ the set of customers, such that $V = D \cup C$ and $D \cap C = \emptyset$. Depot $k \in D$ has at most m_k identical vehicles available, and each customer $i \in C$ has demand $q_i > 0$. Let $c_{ij} \geq 0$ denote the travel cost between nodes i and j , and let $V_k = C \cup \{k\}$ be the customer set together with depot k . Define depot-indexed binary decision variables

$$x_{ij}^k = \begin{cases} 1, & \text{if a vehicle from depot } k \text{ travels directly from } i \text{ to } j, \\ 0, & \text{otherwise,} \end{cases} \quad \forall k \in D, i \neq j, i, j \in V_k,$$

and assignment variables

$$y_{ik} = \begin{cases} 1, & \text{if customer } i \text{ is served by depot } k, \\ 0, & \text{otherwise,} \end{cases} \quad \forall i \in C, k \in D.$$

The multi-depot vehicle routing problem (MDVRP) can be formulated as the

following mixed-integer program:

$$\begin{aligned}
& \min_{x,y} \quad \sum_{k \in D} \sum_{i \in V_k} \sum_{j \in V_k \setminus \{i\}} c_{ij} x_{ij}^k \\
& \text{s.t.} \quad \sum_{k \in D} y_{ik} = 1, \quad \forall i \in C \quad (\text{assign each customer to one depot}) \\
& \quad \sum_{j \in V_k \setminus \{i\}} x_{ij}^k = y_{ik}, \quad \forall i \in C, \forall k \in D \quad (\text{customer departure}) \\
& \quad \sum_{j \in V_k \setminus \{i\}} x_{ji}^k = y_{ik}, \quad \forall i \in C, \forall k \in D \quad (\text{customer arrival}) \\
& \quad \sum_{j \in C} x_{kj}^k = \sum_{j \in C} x_{jk}^k \leq m_k, \quad \forall k \in D \quad (\text{depot flow and route count}) \\
& \quad \sum_{i \in S} \sum_{j \in S \setminus \{i\}} x_{ij}^k \leq |S| - 1, \quad \forall k \in D, \forall S \subseteq C, 2 \leq |S| \leq n - 1 \quad (\text{subtour elimination}) \\
& \quad x_{ij}^k \in \{0, 1\}, \quad \forall k \in D, i \neq j, i, j \in V_k, \\
& \quad y_{ik} \in \{0, 1\}, \quad \forall i \in C, k \in D.
\end{aligned}$$

If vehicle capacities are included, standard capacity constraints, such as MTZ- or flow-based constraints within each depot's routes, can be added to ensure that the total demand on any route does not exceed the vehicle capacity.

2.2 Classical and Learning-Based Solvers

Combinatorial optimization solvers can be broadly grouped into exact methods, relaxation-based methods, and heuristic or metaheuristic methods. In practice, modern state-of-the-art solvers frequently combine these ideas.

2.2.1 Exact Solvers

Exact solvers provide guaranteed optimality, but their computational cost can grow exponentially. Branch and bound (B&B) builds a search tree over partial solutions. At each node, it computes a lower bound, typically through a relaxation such as an LP relaxation. If the bound is worse than the best-known feasible solution, which acts as an upper bound, the node is pruned. B&B is exact, but the search tree can still become exponentially large.

Branch and cut (B&C) augments B&B with cutting planes. When the LP relaxation yields a fractional solution, the solver adds valid inequalities, or cuts, that remove the fractional solution while preserving all integer-feasible solutions. For the TSP, subtour elimination constraints serve as classic cuts. This strategy forms the backbone of solvers such as Concorde [1].

Branch and price (B&P) combines B&B with column generation. Instead of enumerating all variables, which is often exponential, the solver optimizes a restricted master problem and repeatedly generates new variables by solving a pricing subproblem. VRP formulations that use route variables commonly employ this approach.

2.2.2 Relaxation and Decomposition Methods

Relaxation and decomposition methods are widely used to obtain bounds and exploit scalable problem structure. LP relaxation replaces integrality constraints with continuous constraints such as $x \in [0, 1]$, yielding a linear program. LP relaxations provide lower bounds and are standard components in B&B and B&C frameworks, although their fractional solutions may differ substantially from integer-feasible so-

lutions.

Lagrangian relaxation moves hard constraints into the objective function using multipliers. This transformation can produce decomposable subproblems and often yields strong bounds, especially for routing and scheduling problems. Benders decomposition instead splits variables into a master problem and a subproblem. The subproblem generates Benders cuts that iteratively refine the master problem, making the approach effective for problems with naturally separable structure.

2.2.3 Heuristics and Metaheuristics

Heuristics and metaheuristics are designed to obtain high-quality feasible solutions quickly. Local search iteratively improves a feasible solution through small neighborhood moves. In routing problems, common operations include 2-opt, 3-opt, relocate, swap, and cross-exchange. Although local search is fast and effective, it is susceptible to local minima.

Metaheuristics are higher-level strategies designed to escape local minima or explore the solution space more broadly. Prominent examples include simulated annealing (SA), which occasionally accepts worse moves according to a temperature schedule; tabu search, which uses memory to avoid cycling back to recently visited solutions; genetic algorithms (GA), which maintain a population of solutions through crossover and mutation; and ant colony optimization (ACO), which constructs solutions probabilistically based on pheromone trails. These methods do not guarantee optimality but are highly competitive for large instances.

2.2.4 Learning-Assisted Solvers

Recent neural CO research often integrates traditional solvers in two distinct roles. First, exact or high-quality heuristic solutions can serve as teachers or oracles, providing ground-truth labels for supervised learning. Second, classical methods can act as decoding or refinement steps: neural models generate a soft solution, such as a probabilistic heatmap, which is subsequently transformed into a feasible hard solution using greedy decoding, local search such as 2-opt, or other solver components.

In summary, exact solvers provide optimality guarantees at high computational cost, relaxations and decompositions offer scalable bounds, and heuristics provide strong solutions rapidly. Modern NCO systems frequently combine all three, aiming to address the limitations of traditional solvers through data-driven advantages.

2.3 Neural Network Methodologies

2.3.1 Graph Neural Networks

Graph neural networks (GNNs) are neural architectures designed to process data defined on graphs by repeatedly performing message passing. Each node updates its representation by aggregating transformed features from its neighbors. In each layer, a generic GNN computes

$$h_i^{\ell+1} = \phi\left(h_i^\ell, \text{AGG}_{j \in \mathcal{N}(i)} \psi(h_i^\ell, h_j^\ell, e_{ij})\right),$$

where h_i^ℓ is the node embedding at layer ℓ , e_{ij} is an optional edge feature, and AGG is a permutation-invariant operator such as summation or mean aggregation.

In neural combinatorial optimization, classical backbones such as graph convolutional networks (GCNs) [6] and graph attention networks (GATs) [10] primarily produce node embeddings. This is sufficient when the target variable is node-based, such as predicting whether a node is selected in the maximum independent set (MIS). However, for routing problems such as the TSP, the key latent variable is often edge-based, indicating whether an edge belongs to the tour. Consequently, a standard node-centric GNN requires additional structural mechanisms to reliably represent and predict edge decisions.

2.3.2 Anisotropic Graph Neural Networks

Anisotropic graph neural networks (AGNNs) extend traditional message passing by maintaining both node and edge embeddings and by using edge gates so that information flow depends on the current edge state. This architecture is especially useful for diffusion-style denoising networks, where the model ingests noisy variables x_t , such as node states for MIS or edge states for TSP, and predicts the clean variables x_0 .

Let h_i^ℓ and e_{ij}^ℓ denote node and edge features at layer ℓ , and let t be a sinusoidal timestep embedding. A standard AGNN layer used in diffusion backbones first performs an anisotropic edge update:

$$\hat{e}_{ij}^{\ell+1} = P^\ell e_{ij}^\ell + Q^\ell h_i^\ell + R^\ell h_j^\ell.$$

The edges are then refined with residual multilayer perceptrons (MLPs) and timestep

conditioning:

$$e_{ij}^{\ell+1} = e_{ij}^{\ell} + \text{MLP}_e(\text{BN}(\hat{e}_{ij}^{\ell+1})) + \text{MLP}_t(t).$$

The node update then uses edge-gated messages:

$$h_i^{\ell+1} = h_i^{\ell} + \alpha \left(\text{BN} \left(U^{\ell} h_i^{\ell} + \sum_{j \in \mathcal{N}(i)} \sigma(\hat{e}_{ij}^{\ell+1}) \odot V^{\ell} h_j^{\ell} \right) \right).$$

Here $\sigma(\cdot)$ is a sigmoid gate and $\odot$ is the Hadamard product. Intuitively, each neighbor’s message is filtered by the current edge embedding, allowing the network to emphasize or suppress interactions in a direction- and edge-dependent manner.

Because AGNNs jointly model nodes and edges, they naturally support optimization tasks that require edge-variable prediction, such as the TSP. For these problems, e_{ij}^0 is initialized from the noisy edge variables x_t , and h_i^0 is set to positional or sinusoidal node features. The final network head then predicts denoised edge probabilities or values.

2.4 Diffusion Models for COP

2.4.1 DIFUSCO

DIFUSCO (Diffusion for Combinatorial Optimization) solves CO problems by casting a feasible solution as a binary structure, learning a diffusion-style denoising process to transform random noise into that structure, and finally applying a classical decoder, such as greedy decoding followed by 2-opt, to yield a valid tour or route [9].

Representation

DIFUSCO represents the target solution as a binary variable $x_0 \in \{0, 1\}^N$. For TSP, x_0 typically corresponds to edge decisions, indicating whether an edge is included in the tour. Thus, N scales with the number of candidate edges.

Forward Process

A Markov chain gradually corrupts the optimal solution x_0 into random noise x_T using a variance schedule $\{\beta_t\}_{t=1}^T$. In discrete diffusion, the solution x is converted to a one-hot representation $\tilde{x} \in \{0, 1\}^{N \times 2}$ and subjected to a categorical transition:

$$q(x_t \mid x_{t-1}) = \text{Cat}(x_t; p = \tilde{x}_{t-1} Q_t), \quad Q_t = \begin{pmatrix} 1 - \beta_t & \beta_t \\ \beta_t & 1 - \beta_t \end{pmatrix}.$$

As $\prod_{t=1}^T (1 - \beta_t) \approx 0$, the terminal distribution becomes near-uniform.

For continuous diffusion on discrete data, the discrete solution $x_0 \in \{0, 1\}^N$ is rescaled to $\hat{x}_0 \in \{-1, 1\}^N$ and subjected to Gaussian corruption:

$$q(\hat{x}_t \mid \hat{x}_{t-1}) := \mathcal{N}(\hat{x}_t; \sqrt{1 - \beta_t} \hat{x}_{t-1}, \beta_t I).$$

This ensures that $\hat{x}_T$ approaches a standard Gaussian distribution when $\prod_{t=1}^T (1 - \beta_t) \approx 0$.

Reverse Process

DIFUSCO learns the reverse transitions defined by

$$p_\theta(x_0) := \int p_\theta(x_{0:T}) dx_{1:T}, \quad p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t).$$

This is achieved by optimizing the standard variational lower bound, which yields Kullback–Leibler (KL) divergence terms of the form

$$D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t)).$$

The neural network prediction target depends on the diffusion type. In discrete diffusion, the denoiser is trained to predict the clean variable $p_\theta(x_0 | x_t)$. The reverse step is formed by substituting the prediction into the true posterior:

$$p_\theta(x_{t-1} | x_t) \approx q(x_{t-1} | x_t, \hat{x}_0), \quad \hat{x}_0 \sim p_\theta(x_0 | x_t).$$

In continuous diffusion, the denoiser predicts the Gaussian noise ε_t :

$$\varepsilon_t = \frac{\hat{x}_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} = f_\theta(\hat{x}_t, t),$$

which is used to formulate a point estimate of $\hat{x}_0$ inside the Gaussian posterior for $\hat{x}_{t-1}$.

AGNN Denoiser

The conditional model $f_\theta(\cdot, t)$ is implemented using an edge-aware AGNN backbone with timestep conditioning. After sampling the denoised representation to obtain a continuous or probabilistic x_0 , the model applies thresholding or quantization to map the heatmap back to the $\{0, 1\}$ domain. Finally, a non-differentiable decoder, such as greedy edge selection, and optional local improvement heuristics, such as 2-opt, are applied to enforce hard feasibility constraints and minimize the final objective value.

2.5 Reinforcement Learning Fine-Tuning

2.5.1 CADO

While diffusion-based models such as DIFUSCO demonstrate strong performance using supervised learning (SL), this paradigm suffers from a fundamental objective mismatch. In SL, the neural network is trained to imitate optimal ground-truth solutions by minimizing surrogate losses, such as cross-entropy. However, minimizing this imitation loss does not guarantee minimizing the actual decoded solution cost.

CADO (Cost-Aware Diffusion Solvers for Combinatorial Optimization through RL Fine-Tuning) analyzes this objective mismatch through two main deficiencies [11]. The first is decoder-blindness: the SL objective is oblivious to the non-differentiable decoding heuristic f_g , such as greedy search or 2-opt, that projects the continuous heatmap into a discrete feasible solution. The second is cost-blindness: SL prioritizes structural similarity to the ground-truth solution over the actual objective value of the problem. Empirical results in CADO show that structural proximity to an opti-

mal solution, measured by Hamming distance, can have near-zero correlation with the final decoded solution cost.

To address these issues, CADO introduces a reinforcement learning (RL) fine-tuning framework that explicitly aligns the diffusion process with the true CO objective.

2.5.2 MDP Formulation

Instead of relying only on structural imitation, CADO directly optimizes the post-decoded solution cost by formulating the iterative diffusion denoising process as a Markov decision process (MDP). This formulation enables the model to be trained using the REINFORCE algorithm.

Let the MDP be represented by the tuple $(\mathcal{S}, \mathcal{A}, P, \rho_0, R)$. The state $s_t \in \mathcal{S}$, action $a_t \in \mathcal{A}$, and reward function $R(s_t, a_t)$ at timestep t are defined as

$$s_t \triangleq (g, T - t, x_{T-t}), \quad a_t \triangleq x_{t-1},$$

and

$$R(s_t, a_t) \triangleq \begin{cases} -c_g(f_g(x_0)), & \text{if } t = T, \\ 0, & \text{otherwise.} \end{cases}$$

Here, g represents the problem instance, $c_g(\cdot)$ is the true cost function, and $f_g(\cdot)$ is the non-differentiable decoder. By evaluating the reward only at the final step, after the heatmap has been fully decoded, this formulation explicitly incorporates both the decoder’s behavior and the true cost feedback, resolving decoder-blindness and cost-blindness.

2.5.3 Reward Strategies

To stabilize RL training and reduce variance, CADO uses two reward formulation strategies. The standard reward (SR) uses the negative true solution cost directly as the reward signal. This label-free formulation enables the model to learn through trial-and-error exploration and can remain applicable even when labeled solutions are unavailable.

The label-centered reward (LCR) uses information from the SL pre-training dataset without returning to pure imitation. It repurposes the ground-truth solution cost $b_D(g)$ as an instance-specific baseline and defines the reward as the negative optimality gap:

$$R_t = -(c_g(f_g(x_0)) - b_D(g)).$$

Because the baseline $b_D(g)$ depends only on the problem instance and is independent of the policy’s actions, it serves as an unbiased baseline. Thus, the policy gradient remains consistent with true cost minimization even if the dataset labels are computationally suboptimal.

2.5.4 Hybrid Fine-Tuning

Applying RL fine-tuning naively to the large AGNN architectures used in diffusion models can be unstable and memory-intensive. To achieve parameter-efficient and stable adaptation, CADO employs a hybrid fine-tuning (Hybrid-FT) strategy.

Hybrid-FT applies low-rank adaptation (LoRA) to the input layer and most backbone GNN layers. This preserves the robust features learned during SL pre-training while introducing only a small set of trainable low-rank matrices. At the

same time, CADO applies selective layer fine-tuning (Selective-FT), in which all parameters are retrained, to the final GNN layer and the output layer. These layers are most directly responsible for final heatmap generation. The combination supports rapid initial convergence while maintaining sufficient expressive power for cost-aware adaptation.

제 3 장 Methodology

3.1 Overview

This thesis presents a methodology for training and evaluating diffusion-based neural solvers for routing problems. First, a DIFUSCO-style model is trained with supervised denoising on solver-generated labels. Second, the supervised checkpoint is optionally fine-tuned with a CADO-style reinforcement learning objective that directly rewards the decoded solution quality.

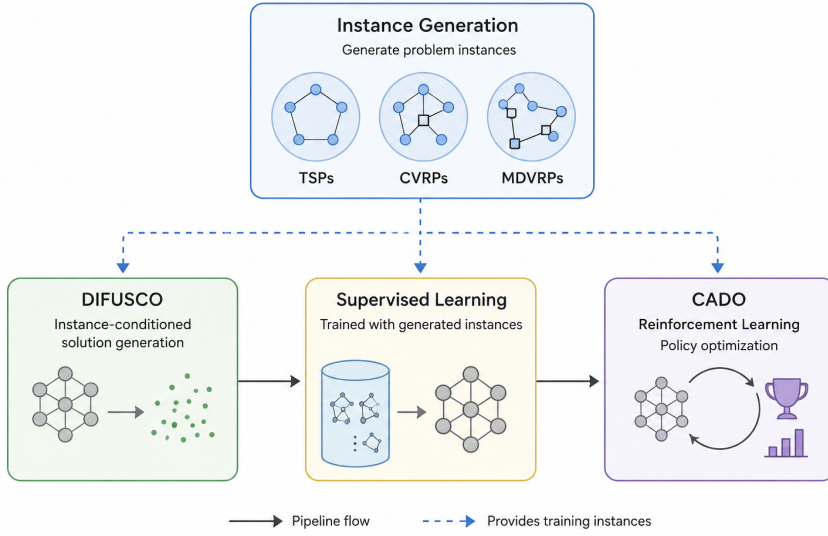


Figure 3.1: Overview of the proposed training and evaluation pipeline for diffusion-based neural solvers. The pipeline consists of supervised denoising training followed by optional reinforcement learning (CADO-style) fine-tuning.

The implementation covers three problem settings. For TSP, the model predicts

a heatmap over tour edges. For CVRP, the same edge-heatmap paradigm is extended to capacitated multi-route solutions. For MDVRP, the implementation focuses on depot-assignment heatmaps as a first step toward full multi-depot routing. This distinction is important: the TSP and CVRP experiments report decoded route length and optimality gap, whereas the MDVRP experiment reports depot assignment accuracy and capacity-violation rate.

3.2 Model Architecture

The core model follows the DIFUSCO design described in section 2.4.1: a categorical diffusion process corrupts binary solution variables, and an edge-aware graph neural network predicts the clean binary structure from a noisy intermediate state. The implementation uses two model sizes. The small model has hidden dimension 128 and 6 AGNN layers, while the paper model has hidden dimension 256 and 12 AGNN layers.

For TSP and CVRP, the binary variable represents edge membership. The graph input contains node coordinates, edge indices, edge distances, and the current noisy edge state x_t . The network returns logits for each edge, and the probability of class 1 is interpreted as the heatmap value indicating that the edge should be selected. CVRP additionally includes customer demands and vehicle capacity during decoding, although capacity is not used directly in the supervised denoising loss.

For MDVRP, the model predicts customer–depot assignment edges rather than full routes. The decoder converts the assignment heatmap into a depot choice for each customer while respecting aggregate depot capacity where possible. This simplified

output space makes MDVRP feasible as an initial extension, but it does not yet solve the complete route construction problem inside each depot.

3.3 Supervised Training

Supervised training uses the ground-truth binary edge labels generated by classical solvers. At each training step, a timestep t is sampled uniformly, the ground-truth label x_0 is corrupted by the categorical diffusion process to obtain x_t , and the model predicts x_0 from (g, x_t, t) . The loss is the cross-entropy between the predicted logits and the ground-truth binary labels:

$$\mathcal{L}_{\text{SL}} = \text{CE}(f_{\theta}(g, x_t, t), x_0).$$

All supervised runs use $T = 1000$ diffusion steps with a linear beta schedule from 10^{-4} to 0.02. Optimization uses AdamW with learning rate 2×10^{-4} , weight decay 10^{-4} , cosine learning-rate decay, and gradient clipping at norm 1.0. The local configuration uses batch size 16, while remote A100 runs use batch size 64. Validation uses 50 denoising steps subsampled from the training diffusion chain with a cosine schedule.

3.4 CADO Fine-Tuning

CADO fine-tuning starts from a supervised DIFUSCO checkpoint and optimizes the decoded solution cost using REINFORCE. During a rollout, the model samples a denoising trajectory with $M_{\text{train}} = 5$ inference steps and accumulates the log-

probabilities of the sampled reverse transitions. After the final heatmap is thresholded or decoded, the reward is computed from the resulting hard solution.

For TSP and CVRP, the standard reward (SR) is the negative decoded solution length. The label-centered reward (LCR) subtracts the ground-truth length and therefore rewards the negative optimality gap. In the implementation, SR uses batch-normalized advantages:

$$A_i = \frac{R_i - \bar{R}}{\text{std}(R) + 10^{-6}}, \quad \mathcal{L}_{\text{RF}} = -\frac{1}{B} \sum_{i=1}^B A_i \log p_{\theta}(\tau_i).$$

The CADO training code also includes a small entropy term based on the sampled log probabilities to encourage exploration.

To reduce memory cost and preserve the supervised representation, CADO uses Hybrid-FT. LoRA with rank 2 is applied to most of the backbone, while the final GNN layer and output head are fully trainable. In the TSP small-model run, this reduced the trainable parameter count from approximately 953K parameters to 178K parameters, or 18.7% of the model.

3.5 Decoding and Evaluation

At evaluation time, the model generates a probabilistic heatmap with a reduced number of denoising steps rather than all $T = 1000$ diffusion steps. The supervised runs use $M_{\text{eval}} = 50$; CADO runs use the value specified by the corresponding configuration or experiment log. TSP decoding selects high-probability edges and applies 2-opt local search. CVRP decoding ranks undirected edge scores by heatmap probability divided by Euclidean distance, greedily adds edges subject to degree,

subtour, and capacity checks, extracts depot-rooted routes, and optionally applies per-route 2-opt. The CVRP validation metric is the percentage optimality gap

$$\text{Gap}(\%) = \frac{L_{\text{pred}} - L_{\text{gt}}}{L_{\text{gt}}} \times 100,$$

where L_{pred} is the decoded route length and L_{gt} is the solver-generated reference length.

MDVRP decoding operates at the assignment level. It averages directed customer–depot edge probabilities, processes customers in descending order of confidence margin, and assigns each customer to the highest-scoring depot with remaining capacity. The reported MDVRP metrics are assignment accuracy and capacity-violation rate.

3.6 Implementation

The codebase separates supervised diffusion solvers and CADO fine-tuning. The `difusco/` package contains supervised TSP, CVRP, and MDVRP implementations, including datasets, model definitions, trainers, decoders, and evaluation entry points. The `cado/` package wraps the corresponding supervised models with rollout and reward functions for REINFORCE fine-tuning. Experiment settings are managed with Hydra under `configs/`, including problem data, model size, diffusion schedule, training settings, and inference settings.

Datasets are generated under `data_generation/`. TSP labels are generated with Concorde, while CVRP and MDVRP labels are generated with PyVRP. Runs are logged to Weights & Biases, and checkpoints are saved under `checkpoints/`. The main experiments use seed 42 for training splits and model initialization, with local

runs on Apple MPS and larger CVRP/MDVRP runs on a VESSL A100 instance.

제 4 장 Experiments

4.1 Experimental Setup

The experiments evaluate whether diffusion-based heatmap solvers can be trained and fine-tuned for increasingly constrained routing problems. TSP is used as the baseline setting, CVRP tests whether the approach remains useful under vehicle-capacity constraints, and MDVRP tests an initial multi-depot assignment extension.

Two compute environments were used. TSP supervised and CADO experiments were run locally on an Apple M4 Pro machine with 24 GB unified memory using the PyTorch MPS backend. Larger CVRP and MDVRP supervised runs were executed remotely on a VESSL A100 instance. All main runs use seed 42. Supervised training uses AdamW with learning rate 2×10^{-4} , weight decay 10^{-4} , cosine learning-rate decay, and gradient clipping at norm 1.0. CADO fine-tuning uses AdamW with learning rate 10^{-4} , no weight decay, LoRA rank 2, one selectively fine-tuned final layer, and $M_{\text{train}} = 5$. The CADO evaluation step count follows the individual experiment configuration.

4.2 Datasets

The datasets consist of synthetic Euclidean routing instances with solver-generated labels. TSP-50 contains 128,000 instances solved by Concorde. CVRP-50 contains single-depot instances with 50 customers generated from the X-style distribution and

solved by PyVRP. The CVRP file used by the main run contains 128,000 labeled instances. The main MDVRP run uses a generated dataset of 64,000 multi-depot instances with 50 customers.

For supervised training, the dataset is split into 90% training and 10% validation. To keep validation cost manageable, each validation pass evaluates a fixed subset: 500 instances for the supervised TSP, CVRP, and MDVRP runs, 100 instances for the short TSP CADO pilot, 200 instances for the extended TSP CADO run, and 100 instances for CVRP CADO.

4.3 Baselines and Metrics

The primary baseline is the supervised DIFUSCO checkpoint trained with cross-entropy on solver-generated labels. CADO fine-tuning is evaluated as a second-stage improvement over that checkpoint. For TSP and CVRP, the main metric is optimality gap:

$$\text{Gap}(\%) = \frac{L_{\text{pred}} - L_{\text{gt}}}{L_{\text{gt}}} \times 100.$$

The decoded length L_{pred} is measured after greedy decoding and 2-opt post-processing. For CVRP, the validation also reports the capacity-violation rate. For MDVRP, the current implementation reports depot assignment accuracy and capacity-violation rate, because full route-cost evaluation for MDVRP is not yet integrated into the training loop.

Table 4.1: Summary of main experimental results. Lower gap is better; higher MDVRP assignment accuracy is better.

Problem	Method	Best metric	Final metric	Notes
TSP-50	DIFUSCO SL	4.46% gap	4.46% gap	small model
TSP-50	CADO, 100 epochs	3.77% gap	3.77% gap	SR, eval subset 100
TSP-50	CADO, 1000 epochs	3.96% gap	4.03% gap	SR, eval subset 200
CVRP-50	DIFUSCO SL	25.96% gap	35.51% gap	paper model, 2-opt
CVRP-50	CADO, 1000 epochs	20.24% gap	47.16% gap	best at epoch 470
MDVRP-50	DIFUSCO SL	81.89% acc.	55.75% acc.	assignment only

4.4 Main Results

The TSP results show that CADO fine-tuning can improve a supervised diffusion solver even with a small model. The supervised TSP-50 baseline reaches a 4.46% validation gap. A 100-epoch CADO run improves this to 3.77%, a 0.69 percentage-point absolute reduction and a 15.4% relative reduction in gap. Extending the run to 1000 epochs gives a best gap of 3.96%, with a final gap of 4.03%. Because the extended run uses a larger validation subset than the 100-epoch pilot, the two CADO numbers should not be compared point-by-point; both indicate that RL fine-tuning improves over the supervised starting point, but the gains are modest and noisy.

The CVRP results are more challenging. The supervised CVRP-50 run with the paper-size model and per-route 2-opt reaches a best validation gap of 25.96% at epoch 10, but the final epoch degrades to 35.51%. CADO fine-tuning improves the best checkpoint to 20.24% at epoch 470, showing that cost-aware fine-tuning can improve decoded CVRP quality. However, the same run collapses later, ending at 47.16% gap. Thus, for CVRP the useful result is the best validation-selected checkpoint, not the final checkpoint.

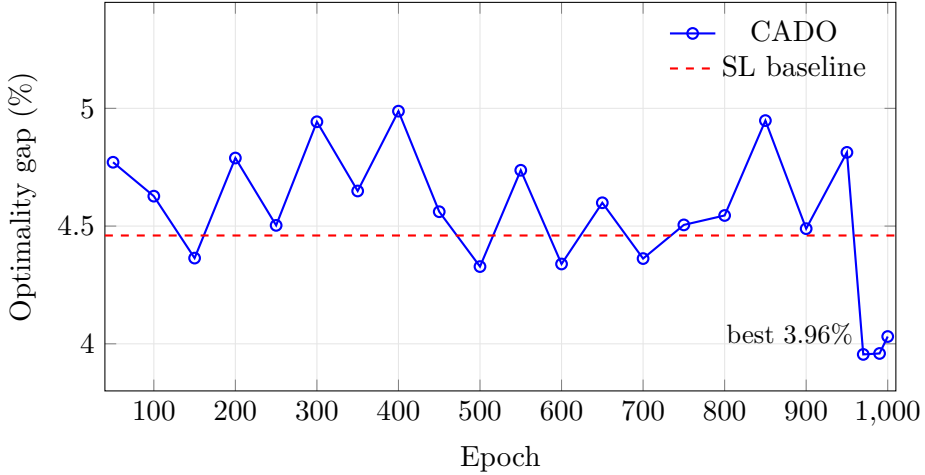


Figure 4.1: TSP-50 validation gap during the extended CADO run. The dashed line shows the supervised checkpoint used to initialize fine-tuning.

The MDVRP experiment is best interpreted as an implementation and diagnostic result rather than a full routing benchmark. The supervised assignment model reaches 81.89% assignment accuracy at the first validation point, while the final model drops to 55.75%. Capacity violations remain at 0% throughout validation. This indicates that the current assignment decoder preserves aggregate capacity but that the supervised diffusion objective does not maintain alignment with depot assignment accuracy over continued training.

4.5 Ablations

Several implementation-level ablations and comparisons were informative.

Effect of 2-opt on CVRP. The CVRP supervised baseline was evaluated with and without per-route 2-opt. Greedy decoding alone reached approximately 29.98% best validation gap, while enabling 2-opt improved the best gap to 25.96%. This

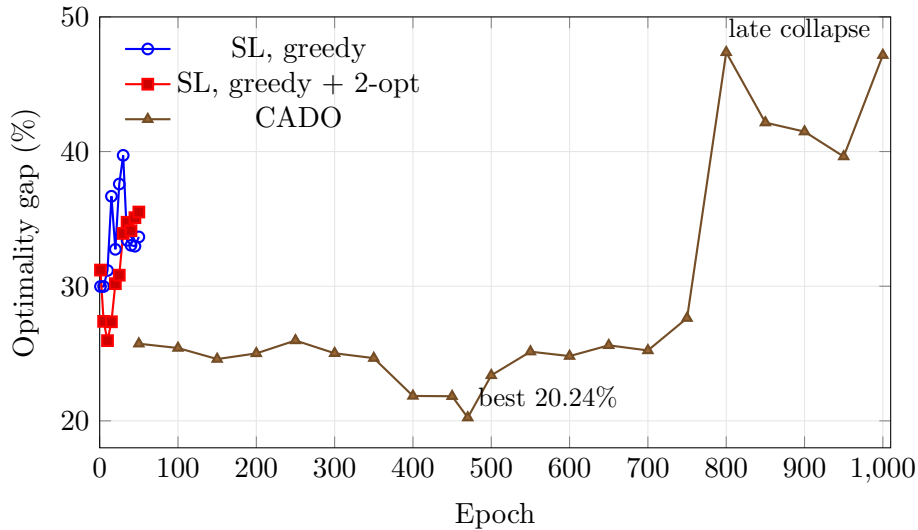


Figure 4.2: CVRP-50 validation gap for supervised decoding variants and CADO fine-tuning. 2-opt improves the best supervised checkpoint, while CADO reaches a lower best gap but later collapses.

confirms that local refinement is an important part of the heatmap-to-route pipeline for CVRP.

Reward mode. Early TSP CADO pilots showed that LCR was ineffective with the weak supervised baseline used in this thesis: the reward produced little usable relative signal. Switching to SR with batch-normalized advantages and learning rate 10^{-4} produced measurable improvements. For CVRP, the main CADO configuration uses LCR, but the run still exhibits strong instability, suggesting that reward scaling and checkpoint selection remain central issues.

Training duration. Longer CADO training is not uniformly beneficial. On TSP, the 1000-epoch run improves the average validation band only slowly and remains noisy. On CVRP, the 1000-epoch run improves until the middle of training and then

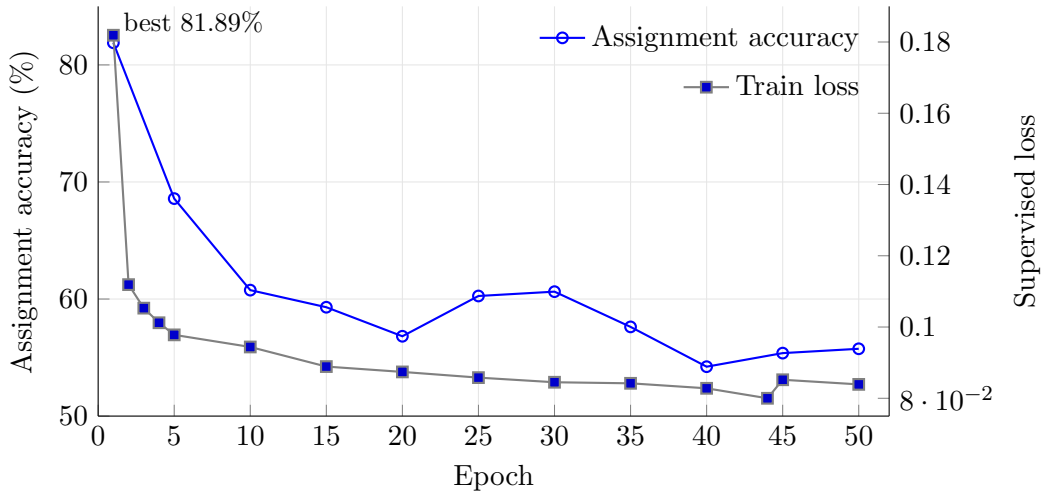


Figure 4.3: MDVRP-50 assignment accuracy and supervised loss. The loss decreases steadily, but assignment accuracy degrades after the first validation point.

collapses. These results support validation-based early stopping rather than selecting the final checkpoint by default.

Parameter-efficient fine-tuning. Hybrid-FT substantially reduces the number of trainable parameters. In the TSP small-model run, only 178K of 953K parameters are trainable. This makes local RL fine-tuning practical, but the very small gradient norms and nearly flat mean log-probabilities observed in the TSP runs suggest limited exploration.

4.6 Discussion

The experiments support three conclusions. First, the heatmap-plus-decoder design can produce feasible routing solutions, and CADO-style fine-tuning can improve the best decoded solution quality over supervised training. This is clearest on TSP

and also visible in CVRP at the best checkpoint.

Second, supervised denoising loss is not a reliable proxy for decoded routing quality. In both CVRP and MDVRP, training loss continues to decrease while validation gap or assignment accuracy worsens. This empirically matches the objective-mismatch argument motivating CADO: the network can become better at predicting labels without becoming better after the non-differentiable decoder.

Third, CADO fine-tuning introduces its own stability challenges. The CVRP CADO run improves substantially at epoch 470 but collapses by epoch 1000. The TSP runs show smaller but noisy improvements. Therefore, the current implementation should be viewed as a successful proof-of-concept for cost-aware fine-tuning rather than a final state-of-the-art solver. Future experiments need larger validation subsets, early stopping, stronger exploration regularization, and full MDVRP route-cost evaluation.

제 5 장 Conclusion

5.1 Summary

This thesis investigated diffusion-based neural solvers for combinatorial optimization, with a focus on routing problems whose feasible solutions must satisfy hard structural constraints. The central motivation was the mismatch between supervised heatmap learning and the final decoded solution quality. A diffusion model may predict labels accurately under a surrogate loss, but the final output is produced only after a non-differentiable decoder enforces problem constraints.

To study this issue, the thesis implemented and evaluated a DIFUSCO-style supervised diffusion solver, together with CADO-style reinforcement learning fine-tuning for TSP and CVRP. The implementation also includes an assignment-level MDVRP extension. TSP and CVRP use edge heatmaps followed by greedy decoding and 2-opt refinement, while MDVRP predicts customer–depot assignments as a first step toward full multi-depot route generation.

The experiments show that CADO fine-tuning can improve the best validation checkpoint over supervised training. On TSP-50, the supervised baseline achieved a 4.46% validation gap, and CADO improved the best observed gap to 3.77% in the 100-epoch pilot. On CVRP-50, supervised training with 2-opt achieved a best gap of 25.96%, while CADO improved the best checkpoint to 20.24%. These results indicate that directly optimizing decoded cost can help align diffusion heatmaps

with routing objectives.

At the same time, the experiments also reveal instability. CVRP CADO collapsed late in training, ending much worse than its best checkpoint. The MDVRP supervised assignment model showed decreasing training loss but deteriorating validation assignment accuracy. These findings reinforce the main thesis claim: for constrained routing problems, the decoder and objective must be treated as central parts of the learning system rather than as afterthoughts.

5.2 Limitations

This work has several limitations. First, the achieved gaps are still far from classical solver quality, especially for CVRP. The current heatmap decoder produces feasible CVRP solutions, but the best CVRP gap remains around 20%, indicating that the learned heatmap and the greedy decoder are not yet strong enough for competitive routing performance.

Second, the validation protocol is limited by subset size. Some CADO experiments use only 100–200 validation instances, so individual best checkpoints may include sampling noise. The reported trends are useful for development, but publication-quality numbers should be computed on larger fixed test sets.

Third, the MDVRP implementation does not yet solve the full multi-depot routing problem. It predicts depot assignments and checks aggregate capacity, but route construction inside each depot is not integrated into the main evaluation loop. Therefore, MDVRP results should be interpreted as assignment-level diagnostics rather than full MDVRP performance.

Finally, CADO fine-tuning is sensitive to reward design, learning rate, training length, and checkpoint selection. The CVRP run demonstrates that a promising mid-training checkpoint can be followed by severe degradation. This makes early stopping and more robust reward normalization necessary for reliable use.

5.3 Future Work

Future work should first strengthen the evaluation protocol. The best TSP and CVRP CADO checkpoints should be re-evaluated on larger fixed validation and test subsets, and all reported comparisons should use identical data splits. Validation-based early stopping should be incorporated directly into the training procedure.

Second, the CVRP decoder and reward should be improved. The current greedy decoder with per-route 2-opt is feasible but leaves substantial gap. Better route construction, stronger local search, or integration with route-level classical heuristics may reduce the gap before and after CADO fine-tuning.

Third, MDVRP should be extended from assignment prediction to full route generation. One natural direction is a two-stage pipeline: first predict depot assignments, then solve or learn a CVRP subproblem for each depot. A more ambitious direction is a unified heatmap over both assignment and routing decisions with a decoder that enforces depot, capacity, and route constraints jointly.

Finally, CADO training should be made more stable. Promising directions include entropy regularization, PPO-style clipped updates, adaptive reward normalization, shorter schedules with early stopping, and reward functions that include the same local-search operations used at evaluation time. These changes would bet-

ter align the training objective with the final decoded solutions while reducing the instability observed in long CVRP fine-tuning runs.

제 A 장 Implementation Details

A.1 Configuration Reference

A.2 Hyperparameter Tables

A.3 Additional Plots

참고 문헌

- [1] David L. Applegate, Robert E. Bixby, Vasek Chvátal, and William J. Cook. The traveling salesman problem: A computational study. In *The Traveling Salesman Problem*. Princeton University Press, 2011.
- [2] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *International Conference on Learning Representations Workshop*, 2017.
- [3] Georges A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.
- [4] Gurobi Optimization, LLC. Gurobi optimizer reference manual. <https://www.gurobi.com>, 2023.
- [5] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12:966–980, 2017.
- [6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [7] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.

- [8] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Courier Corporation, 1998.
- [9] Zhiqing Sun and Yiming Yang. Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems*, 36:3706–3731, 2023.
- [10] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [11] Donghyun Yoon, Hyun Song, Kijung Lee, and Wooseok Lim. Cado: Cost-aware diffusion solvers for combinatorial optimization through rl fine-tuning. In *ICML 2024 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2024.

Abstract

Diffusion Models for Combinatorial Optimization

Janghyeok Choi

Department of Industrial Engineering

The Graduate School

Seoul National University

This thesis studies combinatorial optimization with diffusion-based neural solvers. Decoding heatmaps into feasible solutions under hard constraints is a primary bottleneck; we apply decoder-aware reinforcement fine-tuning in the CADO style to align heatmap generation with the decoding process. Experiments on TSP, CVRP, and MDVRP report optimality gaps, wall-clock cost, and ablations.

Keywords: combinatorial optimization, diffusion models, reinforcement learning, neural combinatorial optimization, vehicle routing

Student Number: 2020-12053

감사의 글